

EMPLOYEE LEAVE SYSTEM TEAM

CLIENT BRIEF

We are a company with multiple employees who need to submit requests to the business, including requests relating to leave. At the moment, employees submit requests through email, WhatsApp and other communication channels. As the company grows, we would like a more organized way of handling these requests and keeping track of decisions that have been made. Employees should have better visibility of requests they have submitted, while the appropriate people within the company should be able to review and manage them. We are open to your team's recommendations on how the system should work.

Team:

Ruby – Team Lead
Fhulufhedzani
Jeremia
Pertunia
Damian

1. The problem + people affected

The client has employees who need to submit requests, including leave. Right now those go through informal channels. That creates three problems: no single place that holds the request, employees cannot easily see status, and reviewers have no clear queue. Decisions can sit in a chat nobody else can find.

Employees chase people for status. Managers get fragmented work. HR and the business have no audit trail when someone asks “was this approved?”

Problem

- Requests arrive through email, WhatsApp, and other channels
- No single record of the request, status, or decision
- The process does not scale as the company grows

People affected

Who	How they are affected today
Employees	Unsure if a request was seen. No reliable history of what they asked for or what was decided.
Managers	Requests arrive in different inboxes and chats. Hard to track or compare.
HR / business	No trustworthy record for policy, disputes, or reporting.

Proposed solution + success

We recommend a web application that becomes the official place for these requests. Leave is the first request type because that is the example the client led with and the one that hurts most when it is informal.

Success looks like

One official channel — leave requests are no longer the official process in email or WhatsApp

An employee can answer “what is the status of my request?” without asking anyone

An approver can open the system and see everything waiting on them

Every decision has an outcome, an actor, a time, and a reason

Fewer lost or duplicate requests because there is one source of truth

2. Stakeholders and users

User	Role	Goal	Main actions
Employee	Staff member submitting requests	Get a request recorded and know its status	Log in, submit leave, view own history, cancel pending
Manager	The “appropriate people” who review	Make and record decisions for their team	See pending queue, approve/reject with reason, view team history
Admin	HR / office admin, assists the admin with filtering and validating leave requests, to help the manager with	Keep the system usable as the company grows	Manage users, roles, leave types, basic reporting.

	making the final decision.		

What information do they need?

Employee

Their own requests (type, dates, reason), current status, and the decision outcome (and rejection reason if rejected)

Manager

Pending requests for their team (who submitted, type, dates, reason) and past decisions for the team

Admin

User list, roles, and leave types.

3. System Requirements

A. Functional requirements (by user)

Employee

FR1 — Submit a leave request

- Who: authenticated employee
- Trigger: chooses “new leave request”
- Information required: leave type, start date, end date, reason; optional supporting file (e.g. sick note)
- Success: request saved as Pending; appears in their list; any attached file stored with the request

- Failure: missing required fields or invalid dates → not saved; form shows what to fix. Unsupported file type or oversize file → upload rejected with a clear message

FR2 — View own requests and status

- Who: authenticated employee
- Trigger: opens their request list
- Information shown: each request with status; rejection reason if rejected; any file they attached
- Constraint: employee A cannot see employee B's requests or files

FR3 — Cancel a pending request

- Who: authenticated employee (owner)
- Trigger: chooses cancel on a request still Pending
- Success: request withdrawn; no longer in the review queue
- Failure: already Approved or Rejected → cancel not allowed

FR4 — View own leave calendar

- Who: authenticated employee
- Trigger: opens their calendar view
- Information shown: their own leave (Pending and Approved) on a calendar
- Constraint: only their own leave; not other employees' leave

Manager

FR5 — View pending queue for their team

- Who: authorised manager
- Trigger: opens the pending queue
- Information shown: pending requests for their team, including any attached files
- Constraint: only their team's requests

FR6 — Approve a pending request

- Who: authorised manager

- Trigger: chooses approve on a pending request they may review
- Success: status = Approved; decision stored with identity and time; employee can see it; leave appears on relevant calendars
- Failure: request not Pending → action rejected

FR7 — Reject a pending request

- Who: authorised manager
- Trigger: chooses reject on a pending request they may review
- Information required: rejection reason
- Success: status = Rejected; reason and decision stored; employee can see them
- Failure: no reason → action blocked

FR8 — View team leave calendar

- Who: authorised manager
- Trigger: opens the team calendar view
- Information shown: leave for their team (Pending and Approved) so overlaps and conflicts are visible
- Constraint: team scope only — not the whole organisation unless they are also Admin

FR9 — Add calendar block for specific dates

- Who: Manager
- Trigger: When the user presses the 'block date' tool while in the calendar view with the correct privileges.
- Information shown: the calendar view, and interactable date blocks.
- Constraint: Admin (and higher privilege) only

Admin

FR10 — Manage users and roles

- Who: Admin
- Trigger: opens user management

- Actions: add or update users; assign roles (Employee, Manager, Admin)
- Purpose: keep the system usable as headcount grows

FR11 — Manage leave types and basic reporting

- Who: Admin
- Actions: maintain leave types; view basic reporting on requests and decisions
- Purpose: configuration and visibility without changing core code

FR12 — View organisation leave calendar

- Who: Admin
- Trigger: opens the organisation calendar view
- Information shown: leave across the organisation so wider overlap can be managed
- Constraint: Admin (and higher privilege) only

FR13 — Add calendar block for specific dates

- Who: Admin
- Trigger: When the user presses the ‘block date’ tool while in the calendar view with the correct privileges.
- Information shown: the calendar view, and interactable date blocks.
- Constraint: Admin (and higher privilege) only

B. Non-functional requirements

- Security — unauthorised users cannot access request data or files; actions and calendar scope limited by role
- Usability — usable on laptop and phone; calendar readable on both
- Validation / errors — clear messages for form errors and failed uploads; no silent failures
- Reliability — submitted requests and attached files remain after refresh
- Auditability — a recorded decision is not silently overwritten; file stays linked to the request

C. Assumptions

These assumptions give version one a clear boundary. The client brief is open to recommendations; the items below are deliberate product decisions.

Assumption	Reason
Web app with login	Shared, persistent record. Informal channels cannot provide that.
Three roles: Employee, Manager, Admin	Matches submitters, reviewers, and someone to keep the system usable as the company grows.
Leave is the first request type	The example the client led with and the case that hurts most when informal.
Employee's manager is the default approver	Simplest model that still matches "the appropriate people."
System is the official channel	Requests start here — not in email or WhatsApp.
Decision record is protected	An employee can cancel only while the request is still Pending.
Optional file attachment on submit	Supporting evidence (e.g. sick note) belongs with the request, not in a chat. Solves the same "scattered record" problem for documents.
Calendar views by role	Employee sees own leave (visibility). Manager sees team calendar (manage overlap). Admin sees organisation calendar. Supports "review and manage" as the company grows.

4. User Stories and Acceptance Criteria

Story 1 — Submit a leave request

As an employee, I want to submit a leave request with dates, a reason, and an optional supporting file, so that the request and any evidence are recorded in one official place.

Acceptance

Given I am logged in as an employee, when I submit valid type, dates, reason, and optionally a supported file, then the request is saved as Pending, appears in my list, and any file is stored with it. Given required fields are missing, dates are invalid, the file is unsupported, **or the dates overlap a blocked date**, then the request is not saved and I see an error I can fix.

Story 2 — Review and decide

As a manager, I want to approve or reject a pending request (and see any attached file), so that the employee and the company have a clear, recorded decision.

Acceptance

Given a pending request I may review, when I approve it, then status is Approved, decision is stored, the employee can see it, and the leave appears on the team calendar. Given I reject without a reason, the action is blocked. With a reason, status is Rejected and the reason is stored and visible to the employee.

Story 3 — Calendar visibility

As an employee, I want to see my own leave on a calendar so that I have clear visibility of what I have requested and what was approved. As a manager, I want to see my team's leave on a calendar so that I can spot overlap and manage coverage before I approve.

Acceptance

Given I am an employee, when I open the calendar, then I only see my own leave (Pending and Approved). Given I am a manager, when I open the team calendar, then I see leave for my team so overlaps are visible. Admin can open an organisation-wide calendar.

Story 4 — Block dates on the calendar

As a manager, I want to block specific dates for my team so that employees cannot book leave on those days when coverage is required. As an Admin, I want to block organisation-wide dates so that leave cannot be requested on company blackout days.

Acceptance

Given I am a manager, when I block a date for my team, then employees in my team cannot submit leave that overlaps that date; they see a clear message. Given I am an Admin, when I block an organisation date, then no employee can submit leave that overlaps that date. Blocked dates are visible on the relevant calendars.

Story 5 — Cancel a pending request

As an employee, I want to cancel a leave request while it is still pending, so that I can withdraw it before a decision is made.

Acceptance

Given my request is Pending, when I cancel it, then it is withdrawn and no longer in the manager's queue.

Given that it is already Approved or Rejected, when I try to cancel, then the action is not allowed.

Story 6 — View own request history

As an employee, I want to see a list of my past and current requests with status (and rejection reason, if any), so that I have a clear record without asking anyone.

Acceptance

Given I am logged in, when I open my requests, then I only see my own requests, with current status and any attached files.

Given another employee's request exists, when I am logged in as me, then I cannot see it.

Story 7 — Manage users and roles (Admin)

As an Admin, I want to add users and assign roles, so the system stays usable as the company grows.

Acceptance

Given that I am Admin, when I add a user, and assign Employee or Manager, then that person can log in with correct access.

Given I am not Admin, when I try to open user management (if visible), then I cannot.

5.) User Flow

Employee — submit leave

Log in → new leave request → enter type, dates, reason → optionally attach file → system validates

Valid and dates not blocked → Pending (file stored) → appears in list and own calendar

Invalid form, bad file, or blocked dates → clear errors → employee corrects or picks other dates

Employee can open own calendar anytime to see Pending/Approved leave and blocked dates that apply to them. Cancel is allowed only while status is Pending.

Manager — decide and manage team calendar

Open pending queue → open a request (details + any file) → optionally check team calendar for overlap → Approve or Reject?

Approve → Approved; who/when stored; leave shows on calendars

Reject → reason required (else blocked) → Rejected; employee sees reason

Block team dates: open team calendar → choose date(s) → save. Team members cannot submit leave overlapping those dates; blocked dates show on the team calendar.

Admin — configuration, organisation calendar, blackouts

Manage users: open user management → add/update user → assign role (Employee, Manager, Admin) → that person can log in with the right access

Maintain leave types and view basic reporting on requests and decisions

Organisation calendar: open org calendar → see leave across the organisation

Block organisation dates: choose date(s) organisation-wide → save. No employee can submit leave overlapping those dates; blocked dates show on relevant calendars.

Final checklist

- Problem Understanding
- People Affected by the Problem
- Proposed Solution
- Success Criteria
- Stakeholders and Users
- User Roles and Goals
- Functional Requirements
- Non-Functional Requirements
- Assumptions
- User Stories
- Acceptance Criteria
- Edge Cases and Important Business Rules
- User Flows